

2a. The two GenAI programs I used were:

- Claude
- ChatGPT

b. I accessed Claude through the Claude AI website on my laptop.

I accessed ChatGPT through the ChatGPT website on my laptop.

c. The prompt I used was:

"Write a program using C++. The program will prioritize emails for a busy company CEO. You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status. You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO. You must create objects from scratch. Do not include pre-existing heap modules."

d. Claude

```
// CEO Email Prioritizer
// Uses a hand-written, list-based (dynamic array) MaxHeap as a priority queue.
// No STL containers or pre-existing heap modules are used for the heap itself.
//
// Build: g++ -std=c++11 -o ceo_email ceo_email.cpp
// Run: ./ceo_email testfile.txt (or run with no args and enter the filename)

#include <iostream>
#include <fstream>
#include <string>

using namespace std;

// -----
// Email object
// -----
class Email {
public:
    string sender; // sender category
    string subject;
    string date; // MM-DD-YYYY
    int priority; // derived from sender category (higher = read sooner)
    int dateKey; // YYYYMMDD, for easy comparison
    long seq; // arrival order, used as a final deterministic tie-breaker

    Email() : sender(""), subject(""), date(""), priority(0), dateKey(0), seq(0) {}

    Email(const string& s, const string& subj, const string& d, long sequence)
        : sender(s), subject(subj), date(d), seq(sequence) {
        priority = categoryPriority(s);
        dateKey = makeDateKey(d);
    }
}
```

```

// Boss > Subordinate > Peer > ImportantPerson > OtherPerson
static int categoryPriority(const string& c) {
    if (c == "Boss")        return 5;
    if (c == "Subordinate")  return 4;
    if (c == "Peer")         return 3;
    if (c == "ImportantPerson") return 2;
    return 1;                // OtherPerson
}

// "MM-DD-YYYY" -> YYYYMMDD
static int makeDateKey(const string& d) {
    if (d.size() < 10) return 0;
    int mm = (d[0] - '0') * 10 + (d[1] - '0');
    int dd = (d[3] - '0') * 10 + (d[4] - '0');
    int yyyy = (d[6] - '0') * 1000 + (d[7] - '0') * 100 +
        (d[8] - '0') * 10 + (d[9] - '0');
    return yyyy * 10000 + mm * 100 + dd;
}

// Returns true if this email should be read BEFORE 'other'.
bool higherPriorityThan(const Email& other) const {
    if (priority != other.priority) return priority > other.priority;
    if (dateKey != other.dateKey) return dateKey > other.dateKey; // newest first
    return seq < other.seq; // earlier arrival first
}
};

// -----
// MaxHeap backed by a list (dynamic array that doubles when full)
// -----
class MaxHeap {
private:
    Email* list; // the underlying list
    int count; // number of elements in the heap
    int capacity; // allocated size of list

    static int parent(int i) { return (i - 1) / 2; }
    static int left(int i) { return 2 * i + 1; }
    static int right(int i) { return 2 * i + 2; }

    void swapAt(int a, int b) {
        Email tmp = list[a];
        list[a] = list[b];
        list[b] = tmp;
    }

    void grow() {
        int newCap = capacity * 2;
        Email* bigger = new Email[newCap];
        for (int i = 0; i < count; i++) bigger[i] = list[i];
        delete[] list;
        list = bigger;
        capacity = newCap;
    }
}

```

```

// Move element at i up until heap property is restored
void siftUp(int i) {
    while (i > 0 && list[i].higherPriorityThan(list[parent(i)]) {
        swapAt(i, parent(i));
        i = parent(i);
    }
}

// Move element at i down until heap property is restored
void siftDown(int i) {
    while (true) {
        int l = left(i), r = right(i), largest = i;
        if (l < count && list[l].higherPriorityThan(list[largest])) largest = l;
        if (r < count && list[r].higherPriorityThan(list[largest])) largest = r;
        if (largest == i) break;
        swapAt(i, largest);
        i = largest;
    }
}

// Disallow copying (owns raw memory)
MaxHeap(const MaxHeap&);
MaxHeap& operator=(const MaxHeap&);

```

public:

```

MaxHeap() : count(0), capacity(16) { list = new Email[capacity]; }
~MaxHeap() { delete[] list; }

```

```

bool isEmpty() const { return count == 0; }
int size() const { return count; }

```

```

void insert(const Email& e) {
    if (count == capacity) grow();
    list[count] = e;
    siftUp(count);
    count++;
}

```

```

// Look at the highest-priority email without removing it
bool peekMax(Email& out) const {
    if (count == 0) return false;
    out = list[0];
    return true;
}

```

```

// Remove and return the highest-priority email
bool extractMax(Email& out) {
    if (count == 0) return false;
    out = list[0];
    list[0] = list[count - 1];
    count--;
    if (count > 0) siftDown(0);
    return true;
}

```

```

    }
};

// -----
// Helpers
// -----
static string trim(const string& s) {
    size_t a = 0, b = s.size();
    while (a < b && (s[a] == ' ' || s[a] == '\t' || s[a] == '\r' || s[a] == '\n')) a++;
    while (b > a && (s[b - 1] == ' ' || s[b - 1] == '\t' || s[b - 1] == '\r' || s[b - 1] == '\n')) b--;
    return s.substr(a, b - a);
}

// Parse "<sender>, <subject>, <date>" (the text after "EMAIL ")
static bool parseEmailFields(const string& rest, string& sender, string& subject, string& date) {
    size_t c1 = rest.find(',');
    if (c1 == string::npos) return false;
    size_t c2 = rest.find(',', c1 + 1);
    if (c2 == string::npos) return false;
    sender = trim(rest.substr(0, c1));
    subject = trim(rest.substr(c1 + 1, c2 - c1 - 1));
    date = trim(rest.substr(c2 + 1));
    return true;
}

// -----
// Main
// -----
int main(int argc, char* argv[]) {
    string filename;
    if (argc > 1) {
        filename = argv[1];
    } else {
        cout << "Enter test file name: ";
        getline(cin, filename);
    }

    ifstream in(filename.c_str());
    if (!in) {
        cerr << "Error: could not open file '" << filename << "'" << endl;
        return 1;
    }

    MaxHeap inbox;
    long nextSeq = 0;
    string line;

    while (getline(in, line)) {
        line = trim(line);
        if (line.empty()) continue;

        if (line.compare(0, 6, "EMAIL ") == 0) {
            string sender, subject, date;
            if (parseEmailFields(line.substr(6), sender, subject, date)) {

```

```

        inbox.insert(Email(sender, subject, date, nextSeq++));
    } else {
        cerr << "Warning: malformed EMAIL command: " << line << endl;
    }
}
else if (line == "NEXT") {
    Email e;
    if (inbox.peekMax(e)) {
        cout << "Sender: " << e.sender << endl;
        cout << "Subject: " << e.subject << endl;
        cout << "Date: " << e.date << endl;
        cout << endl;
    } else {
        cout << "No unread emails." << endl << endl;
    }
}
else if (line == "READ") {
    Email e;
    if (!inbox.extractMax(e)) {
        cout << "No unread emails to mark as read." << endl << endl;
    }
}
else if (line == "COUNT") {
    cout << "Unread emails: " << inbox.size() << endl << endl;
}
else {
    cerr << "Warning: unknown command: " << line << endl;
}
}

return 0;
}

```

Chat GPT

```

#include <iostream>
#include <string>
using namespace std;

// -----
// Email Class
// -----
class Email {
private:
    string sender;
    string subject;
    int priority;

public:
    Email() {
        sender = "";
        subject = "";
        priority = 0;
    }
}

```

```

    }

    Email(string s, string sub, int p) {
        sender = s;
        subject = sub;
        priority = p;
    }

    int getPriority() {
        return priority;
    }

    string getSender() {
        return sender;
    }

    string getSubject() {
        return subject;
    }

    void display() {
        cout << "Priority: " << priority << endl;
        cout << "From: " << sender << endl;
        cout << "Subject: " << subject << endl;
        cout << "-----" << endl;
    }
};

// -----
// MaxHeap Class
// -----
class MaxHeap {
private:
    Email* heap;
    int capacity;
    int size;

    // Move an element upward
    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;

            if (heap[index].getPriority() <= heap[parent].getPriority()) {
                break;
            }

            // Swap child and parent
            Email temp = heap[index];
            heap[index] = heap[parent];
            heap[parent] = temp;

            index = parent;
        }
    }
}

```

```

// Move an element downward
void heapifyDown(int index) {
    while (true) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        // Check left child
        if (left < size &&
            heap[left].getPriority() > heap[largest].getPriority()) {
            largest = left;
        }

        // Check right child
        if (right < size &&
            heap[right].getPriority() > heap[largest].getPriority()) {
            largest = right;
        }

        // If parent is already the largest, stop
        if (largest == index) {
            break;
        }

        // Swap
        Email temp = heap[index];
        heap[index] = heap[largest];
        heap[largest] = temp;

        index = largest;
    }
}

public:
    // Constructor
    MaxHeap(int maxSize) {
        capacity = maxSize;
        size = 0;
        heap = new Email[capacity];
    }

    // Destructor
    ~MaxHeap() {
        delete[] heap;
    }

    // Add an email to the heap
    void insert(Email email) {
        if (size == capacity) {
            cout << "The email queue is full." << endl;
            return;
        }
    }

```

```

    heap[size] = email;

    // Move the new email into the correct position
    heapifyUp(size);

    size++;
}

// Remove and return the highest-priority email
Email removeMax() {
    if (size == 0) {
        cout << "There are no emails in the queue." << endl;
        return Email();
    }

    Email highestPriority = heap[0];

    // Move the last element to the root
    heap[0] = heap[size - 1];
    size--;

    // Restore MaxHeap
    if (size > 0) {
        heapifyDown(0);
    }

    return highestPriority;
}

// Check if heap is empty
bool isEmpty() {
    return size == 0;
}

// Display the current heap
void displayHeap() {
    for (int i = 0; i < size; i++) {
        cout << "Priority: " << heap[i].getPriority()
              << " | From: " << heap[i].getSender()
              << " | Subject: " << heap[i].getSubject()
              << endl;
    }
}

};

// -----
// Main Program
// -----

int main() {

    MaxHeap emailQueue(10);

    // Create emails
    Email email1(

```

```

        "boss@company.com",
        "Important company meeting",
        10
    );

    Email email2(
        "client@company.com",
        "Urgent client request",
        8
    );

    Email email3(
        "employee@company.com",
        "Vacation request",
        3
    );

    Email email4(
        "finance@company.com",
        "Quarterly financial report",
        9
    );

    Email email5(
        "marketing@company.com",
        "New marketing ideas",
        5
    );

    // Add emails to the MaxHeap
    emailQueue.insert(email1);
    emailQueue.insert(email2);
    emailQueue.insert(email3);
    emailQueue.insert(email4);
    emailQueue.insert(email5);

    cout << "Emails currently in the priority queue:" << endl;
    cout << "===== " << endl;

    emailQueue.displayHeap();

    cout << endl;
    cout << "Processing emails by priority:" << endl;
    cout << "===== " << endl;

    // Process emails from highest priority to lowest
    while (!emailQueue.isEmpty()) {
        Email currentEmail = emailQueue.removeMax();
        currentEmail.display();
    }

    return 0;
}

```

e. Correctness:

Claude: The program correctly implements a MaxHeap from scratch and supports inserting, viewing, and removing emails. It also compares sender priority, date, and arrival order to determine which email to process first.

ChatGPT: The program correctly implements the basic MaxHeap operations, including insertion and removal. However, it uses hard-coded emails and priorities instead of accepting email information as input.

- Execution Time:

Both programs use a MaxHeap, so insertion and removal take $O(\log n)$ time, while accessing the highest-priority email takes $O(1)$ time. Claude also has a dynamic resizing operation that can take $O(n)$ when the heap grows.

- Space Complexity

Both programs use $O(n)$ space for storing emails. Claude dynamically increases its array size when needed, while ChatGPT uses a fixed array with a capacity of 10.

- Maintainability

Claude: The code is more maintainable because it separates email processing, priority comparison, heap operations, and input parsing into different functions and classes.

ChatGPT: The code is relatively simple and easy to understand, but the hard-coded emails and fixed heap capacity make it less flexible and harder to modify for different inputs.

f. I selected the Claude program as the basis for my assignment. I selected it because it was closer to the required functionality and required fewer major changes. Claude's program already included file input, email parsing, sender-category priorities, date comparisons, tie-breaking, and the required heap operations. In comparison, the ChatGPT program demonstrated the basic MaxHeap concept but used hard-coded test data and did not process the assignment's input commands. It also did not use email dates when determining priority. Choosing the Claude program allowed me to focus on improving and adapting an already functional solution rather than rebuilding most of the program.

g.

- Correctness: The original output did not match the assignment's required format. I changed it to print "There are N emails to read." for COUNT and a "Next email:" header

for NEXT, and removed the extra blank lines, then confirmed the result matches the sample output exactly. I added validation so an EMAIL line is accepted only if the sender is one of the five valid categories and the date is a valid MM-DD-YYYY (correct dashes, digits, month 1–12, day 1–31). Invalid lines are reported and ignored. I also added messages for NEXT and READ on an empty inbox, and tested two NEXTs in a row, two READs in a row, and Windows line endings.

- Execution time: I kept the MaxHeap, so insert and remove are $O(\log n)$ and NEXT is $O(1)$, instead of $O(n)$ per operation with a linear search. Each date is converted once to an integer (YYYYMMDD) when the email is created, so each heap comparison is an integer compare instead of string parsing.
- Space complexity: I kept the doubling array, which gives $O(n)$ space and amortized $O(1)$ insertion, with no maximum number of emails. I disabled copying of the MaxHeap (= delete) so the raw array can't be accidentally copied and double-freed.
- Maintainability: The assignment requires objects, not functions. I moved the trimming, parsing and validation helpers and all command handling out of main() and into a new EmailInbox class, so main() only creates the object and runs it. The priority rules remain in one method, Email::categoryPriority. I added a prologue and comments on every block and line, with tags showing whether the code came from Claude, from Claude and me, or from me.